

iptables (简体中文)

翻译状态：本文是 [Iptables](#) 的[翻译](#)。上次翻译日期：2015-10-17。如果英文版本有所[更改](#) (<https://wiki.archlinux.org/index.php?title=Iptables&diff=0&oldid=266924>)，则您可以帮助同步翻译。

iptables 是一个配置 Linux 内核 [防火墙](#) 的命令行工具，是 [netfilter](#) 项目的一部分。术语 *iptables* 也经常代指该内核级防火墙。*iptables* 可以直接配置，也可以通过许多 [前端](#)^{[[断开的链接](#)：无效的部分]} 和 [图形界面](#)^{[[断开的链接](#)：无效的部分]} 配置。*iptables* 用于 [ipv4](#)，*ip6tables* 用于 [ipv6](#)。

[nftables](#) 已经包含在 [Linux kernel 3.13](#) (http://www.phoronix.com/scan.php?page=news_item&px=MTQ5MDU) 中，以后会取代 *iptables* 成为主要的 Linux 防火墙工具。

Contents

- 安装
- 基本概念
 - 表 (Tables)
 - 链 (Chains)
 - 规则 (Rules)
 - 遍历链 (Traversing Chains)
 - 模块 (Modules)
- 配置并运行 iptables
 - 从命令行
 - 显示当前规则
 - 重置规则
 - 编辑规则
 - 配置文件
 - 指南
- 日志
 - 限制日志级别
 - 查看记录的数据包
 - 使用 syslog-ng
 - 使用 ulogd
- 参见

相关文章

- [Firewalls](#)
- [Sysctl#TCP/IP stack hardening](#)

安装

Arch Linux 内核已经编译了 iptables 支持。只需要安装在 [official repositories](https://archlinux.org/packages/?name=iptables) 中提供的用户层工具 **iptables** (<https://archlinux.org/packages/?name=iptables>) 包。（**base** (<http://archlinux.org/packages/?name=base>) 组中的 **iproute2** (<https://archlinux.org/packages/?name=iproute2>) 包依赖 iptables，因此系统中已经默认安装了 iptables 包。）

基本概念

iptables 可以检测、修改、转发、重定向和丢弃 IPv4 数据包。过滤 IPv4 数据包的代码已经内置于内核中，并且按照不同的目的被组织成表的集合。表由一组预先定义的链组成，链包含遍历顺序规则。每一条规则包含一个谓词的潜在匹配和相应的动作（称为目标），如果谓词为真，该动作会被执行。也就是说条件匹配。iptables 是用户工具，允许用户使用链和规则。很多新手面对复杂的 linux IP 路由时总是感到气馁，但是，实际上最常用的一些应用案例（NAT 或者基本的网络防火墙）并不是很复杂。

理解 iptables 如何工作的关键是这张图 (http://www.frozentux.net/iptables-tutorial/images/tables_traverse.jpg)。图中在上面的小写字母代表表，在下面的大写字母代表链。从任何网络端口进来的每一个 IP 数据包都要从上到下的穿过这张图。一种常见的困扰是认为 iptables 对从内部端口进入的数据包和从面向互联网端口进入的数据包采取不同的处理方式，相反，iptables 对从任何端口进入的数据包都会采取相同的处理方式。可以定义规则使 iptables 采取不同的方式对待从不同端口进入的数据包。当然一些数据包是用于本地进程的，因此在图中表现为从顶端进入，到 <Local Process> 停止，而另一些数据包是由本地进程生成的，因此在图中表现为从 <Local Process> 发出，一直向下穿过该流程图。一份关于该流程图如何工作的详细解释请参考[这里](http://www.frozentux.net/iptables-tutorial/iptables-tutorial.html#TRAVERSINGOFTABLES) (<http://www.frozentux.net/iptables-tutorial/iptables-tutorial.html#TRAVERSINGOFTABLES>)。

在大多数使用情况下都不会用到 **raw**，**mangle** 和 **security** 表。下图简要描述了网络数据包通过 iptables 的过程：

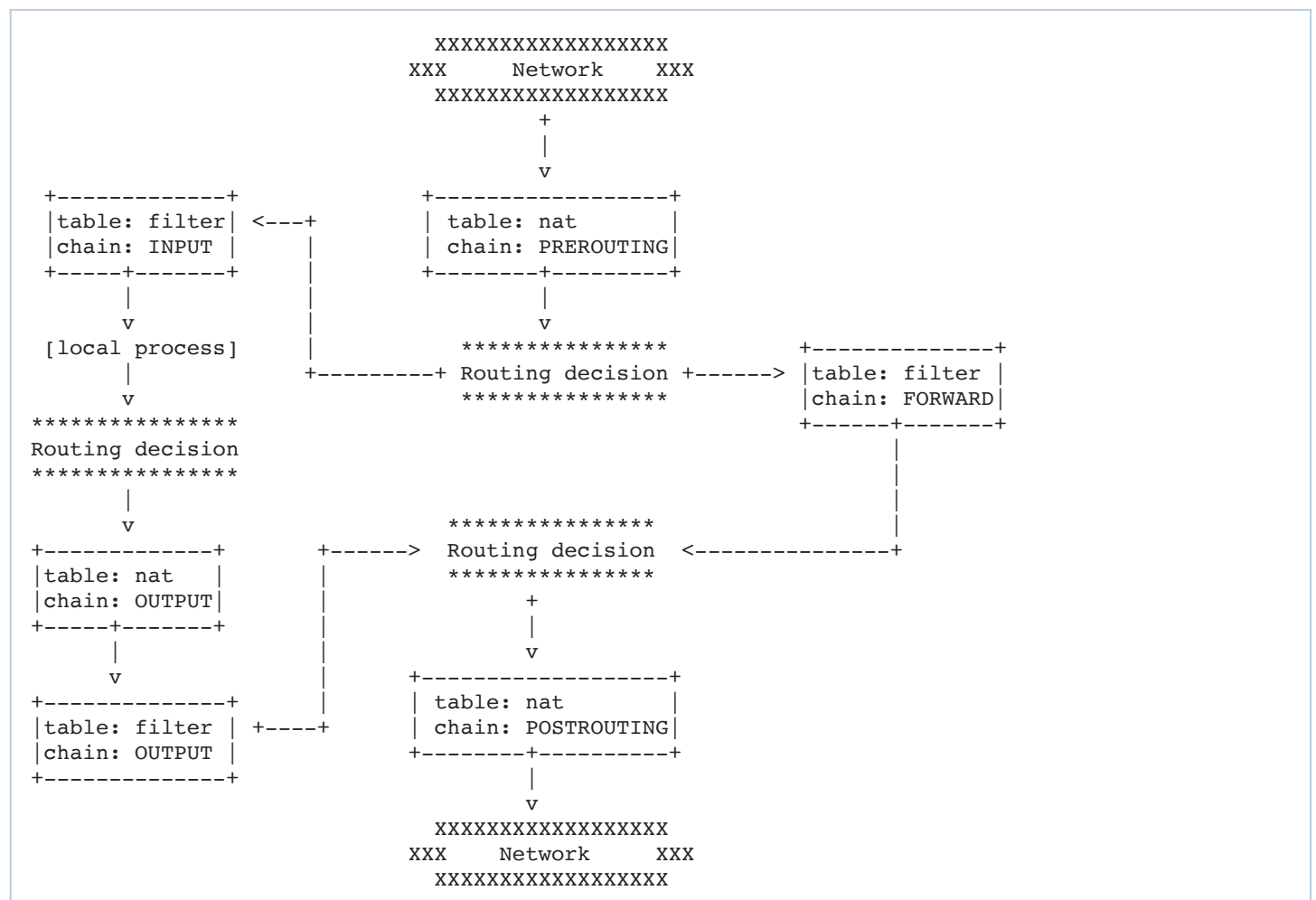


表 (Tables)

iptables 包含 5 张表 (tables) :

1. **raw** 用于配置数据包, **raw** 中的数据包不会被系统跟踪。
2. **filter** 是用于存放所有与防火墙相关操作的默认表。
3. **nat** 用于 [网络地址转换](#) (例如: 端口转发) 。
4. **mangle** 用于对特定数据包的修改 (参考 [损坏数据包](#)) 。
5. **security** 用于 [强制访问控制](#) 网络规则 (例如: SELinux -- 详细信息参考 [该文章 \(http://lwn.net/Articles/267140/\)](http://lwn.net/Articles/267140/)) 。

大部分情况仅需要使用 **filter** 和 **nat**。其他表用于更复杂的情况——包括多路由和路由判定——已经超出了本文介绍的范围。

链 (Chains)

表由链组成, 链是一些按顺序排列的规则列表。默认的 **filter** 表包含 **INPUT**, **OUTPUT** 和 **FORWARD** 3条内建的链, 这3条链作用于数据包过滤过程中的不同时间点, 如该[流程图 \(http://www.frozentux.net/iptables-tutorial/chunkyhtml/images/tables_traverse.jpg\)](http://www.frozentux.net/iptables-tutorial/chunkyhtml/images/tables_traverse.jpg) 所示。 **nat** 表包含 **PREROUTING**, **POSTROUTING** 和 **OUTPUT** 链。

使用 [iptables\(8\) \(https://man.archlinux.org/man/iptables.8\)](https://man.archlinux.org/man/iptables.8) 查看其他表中内建链的描述。

默认情况下, 任何链中都没有规则。可以向链中添加自己想用的规则。链的默认规则通常设置为 **ACCEPT**, 如果想确保任何包都不能通过规则集, 那么可以重置为 **DROP**。默认的规则总是在一条链的最后生效, 所以在默认规则生效前数据包需要通过所有存在的规则。

用户可以加入自己定义的链, 从而使规则集更有效并且易于修改。如何使用自定义链请参考 [Simple stateful firewall](#)。

规则 (Rules)

数据包的过滤基于规则。规则由一个 *目标* (数据包匹配所有条件后的动作) 和很多 *匹配* (导致该规则可以应用的数据包所满足的条件) 指定。一个规则的典型匹配事项是数据包进入的端口 (例如: **eth0** 或者 **eth1**)、数据包的类型 (**ICMP**, **TCP**, 或者 **UDP**) 和数据包的目的地。

目标使用 **-j** 或者 **--jump** 选项指定。目标可以是用户定义的链 (例如, 如果条件匹配, 跳转到之后的用户定义的链, 继续处理)、一个内置的特定目标或者是一个目标扩展。内置目标是 **ACCEPT**, **DROP**, **QUEUE** 和 **RETURN**, 目标扩展是 **REJECT** and **LOG**。如果目标是内置目标, 数据包的命运立刻被决定并且在当前表的数据包的处理过程会停止。如果目标是用户定义的链, 并且数据包成功穿过第二条链, 目标将移动到原始链中的下一个规则。目标扩展可以被终止 (像内置目标一样) 或者不终止 (像用户定义链一样)。详细信息参阅 [iptables-extensions\(8\) \(https://man.archlinux.org/man/iptables-extensions.8\)](https://man.archlinux.org/man/iptables-extensions.8)。

遍历链 (Traversing Chains)

该[流程图 \(http://www.frozentux.net/iptables-tutorial/chunkyhtml/images/tables_traverse.jpg\)](http://www.frozentux.net/iptables-tutorial/chunkyhtml/images/tables_traverse.jpg)描述链了在任何接口上收到的网络数据包是按照怎样的顺序穿过表的交通管制链。第一个路由策略包括决定数据包的目的地是本地主机 (这种情况下, 数据包穿过 **INPUT** 链), 还是其他主机

（数据包穿过 FORWARD 链）；中间的路由策略包括决定给传出的数据包使用那个源地址、分配哪个接口；最后一个路由策略存在是因为先前的 mangle 与 nat 链可能会改变数据包的路由信息。数据包通过路径上的每一条链时，链中的每一条规则按顺序匹配；无论何时匹配了一条规则，相应的 target/jump 动作将会执行。最常用的3个 target 是 ACCEPT，DROP，或者 jump 到用户自定义的链。内置的链有默认的策略，但是用户自定义的链没有默认的策略。在 jump 到的链中，若每一条规则都不能提供完全匹配，那么数据包像[这张图片 \(http://www.frozentux.net/iptables-tutorial/image_s/table_subtraverse.jpg\)](http://www.frozentux.net/iptables-tutorial/image_s/table_subtraverse.jpg)描述的一样返回到调用链。在任何时候，若 DROP target 的规则实现完全匹配，那么被匹配的数据包会被丢弃，不会进行进一步处理。如果一个数据包在链中被 ACCEPT，那么它也会被所有的父链 ACCEPT，并且不再遍历其他父链。然而，要注意的是，数据包还会以正常的方式继续遍历其他表中的其他链。

模块（Modules）

有许多模块可以用来扩展 iptables，例如 connlimit, conntrack, limit 和 recent。这些模块增添了功能，可以进行更复杂的过滤。

配置并运行 iptables

iptables 是一个 [Systemd](#) 服务，因此可以这样启动：

```
# systemctl start iptables
```

但是，除非有 /etc/iptables/iptables.rules 文件，否则服务不会启动，Arch [iptables \(https://archlinux.org/packages/?name=iptables\)](https://archlinux.org/packages/?name=iptables) 包不包含默认的 iptables.rules 文件。因此，第一次启动服务时使用以下命令：

```
# touch /etc/iptables/iptables.rules
# systemctl start iptables
```

或者

```
# cp /etc/iptables/empty.rules /etc/iptables/iptables.rules
# systemctl start iptables
```

和其他服务一样，如果希望启动时自动加载 iptables，必须启用该服务：

```
# systemctl enable iptables
```

从命令行

显示当前规则

使用以下命令查看当前规则和匹配数：

```
# iptables -nvL

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
```

pkts	bytes	target	prot	opt	in	out	source	destination
Chain OUTPUT (policy ACCEPT 0K packets, 0 bytes)								
pkts	bytes	target	prot	opt	in	out	source	destination

上面的结果表明还没有配置规则。没有数据包被阻止。

在输入中添加 `--line-numbers` 选项，可以在列出规则的时候显示行号。这在添加单独的规则的时候很有用。

重置规则

使用这些命令刷新和重置 iptables 到默认状态：

```
# iptables -F
# iptables -X
# iptables -t nat -F
# iptables -t nat -X
# iptables -t mangle -F
# iptables -t mangle -X
# iptables -t raw -F
# iptables -t raw -X
# iptables -t security -F
# iptables -t security -X
# iptables -P INPUT ACCEPT
# iptables -P FORWARD ACCEPT
# iptables -P OUTPUT ACCEPT
```

没有任何参数的 `-F` 命令在当前表中刷新所有链。同样的，`-X` 命令删除表中所有非默认链。

用下面的带 `[chain]` 参数的 `-F` 和 `-X` 命令刷新或删除单独的链。

编辑规则

有两种方式添加规则，一种是在链上附加规则，另一种是将规则插入到链上某个特定位置。这里将讲解这两种方式。

首先，由于电脑不是路由器(unless, of course, it **is a router**)，因此将 FORWARD 链默认的规则由 ACCEPT 改成 DROP。

```
# iptables -P FORWARD DROP
```

警告： 本章节其余部分主要讲解 iptables 规则的语法和其背后的思想，而不是作为一种保护服务器的手段，如果想要提高系统的安全性，参考 [Simple stateful firewall](#) 来获得一个最低限度的 iptables 安全配置，参考 [Security](#) 来提高 Arch Linux 的安全性。

[Dropbox](#) 的局域网同步特性 — [每30秒广播数据包 \(https://isc.sans.edu/port.html?port=17500\)](https://isc.sans.edu/port.html?port=17500)到所有可视的计算机，如果我们碰巧在一个拥有 Dropbox 客户端的局域网中，但是我们不想使用这个特性，那么我们希望拒绝这些数据包。

```
# iptables -A INPUT -p tcp --dport 17500 -j REJECT --reject-with icmp-port-unreachable
```

```
# iptables -nvL --line-numbers
```


Chain INPUT (policy ACCEPT 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	
1	0	0	REJECT	tcp	--	*	*	0.0.0.0/0	0.0.0.0/0	tc
p dpt:17500 reject-with icmp-port-unreachable										
Chain FORWARD (policy DROP 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	

注意：这里使用 REJECT 而不是 DROP，因为 [RFC 1122 3.3.8 \(https://tools.ietf.org/html/rfc1122#page-69\)](https://tools.ietf.org/html/rfc1122#page-69) 要求主机尽可能返回 ICMP 错误而不是丢弃数据包。[这里 \(http://www.chiark.greenend.org.uk/~peterb/network/drop-vs-reject\)](http://www.chiark.greenend.org.uk/~peterb/network/drop-vs-reject) 介绍了为什么倾向于 REJECT 而不是 DROP 数据包。

现在，我们改变对 Dropbox 的看法，并且决定安装其到我们的计算机，我们也希望局域网同步，但是我们的网络只有一个特定的 IP 地址，因此需要使用 -R 参数来替换旧规则，10.0.0.85 是我们的另一个 IP 地址。

```
# iptables -R INPUT 1 -p tcp --dport 17500 ! -s 10.0.0.85 -j REJECT --reject-with icmp-port-unreachable
```

# iptables -nvL --line-numbers										
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	
1	0	0	REJECT	tcp	--	*	*	!10.0.0.85	0.0.0.0/0	tc
p dpt:17500 reject-with icmp-port-unreachable										
Chain FORWARD (policy DROP 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	

现在我们替换了原来的规则，使 10.0.0.85 可以访问计算机 17500 端口。但是我们意识到这条规则是不可升级的。如果友好的 Dropbox 用户想要访问设备上的 17500 端口。我们应该马上允许，而不是在测试过所有防火墙规则后再允许。

因此我们写一条新规则来立即允许受信任的用户。使用 -I 参数来在旧规则前插入一条新规则：

```
# iptables -I INPUT -p tcp --dport 17500 -s 10.0.0.85 -j ACCEPT -m comment --comment "Friendly Dropbox"
```

# iptables -nvL --line-numbers										
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	
1	0	0	ACCEPT	tcp	--	*	*	10.0.0.85	0.0.0.0/0	tc
p dpt:17500 /* Friendly Dropbox */										
2	0	0	REJECT	tcp	--	*	*	!10.0.0.85	0.0.0.0/0	tc
p dpt:17500 reject-with icmp-port-unreachable										
Chain FORWARD (policy DROP 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	

更改第二条规则，使其拒绝 17500 端口的任何数据包：

```
# iptables -R INPUT 2 -p tcp --dport 17500 -j REJECT --reject-with icmp-port-unreachable
```

我们的最终规则如下：

```
# iptables -nvL --line-numbers

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination          tc
1      0      0 ACCEPT      tcp  --  *      *       10.0.0.85            0.0.0.0/0            tc
p dpt:17500 /* Friendly Dropbox */
2      0      0 REJECT      tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tc
p dpt:17500 reject-with icmp-port-unreachable

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
```

配置文件

在 Arch Linux 中，Iptables 规则默认存放在 `/etc/iptables/iptables.rules` 文件中，它们不会被自动加载，因此需要启用 `iptables.service`，服务会在启动时读取文件并且加载规则，或者直接启动服务：

```
# systemctl enable iptables.service
# systemctl start iptables.service
```

ipv6 规则默认保存在 `/etc/iptables/ip6tables.rules`，`ip6tables.service` 服务会使用这个规则，可以用类似的方式启动服务。

注意：iptables 1.4.21-1 包中的 `iptables.service` 和 `ip6tables.service` 文件已经过期了。因为安全原因，自 `systemd 214` 起推荐防火墙在 `network-pre.target` 目标之前启动，这样防火墙就可以在任何网络配置之前运行。等待 iptables 数据包更新，创建深层目录 `/etc/systemd/system/iptables.service.d`，创建 `00-pre-network.conf` 文件，添加如下片段：

```
[Unit]
Before=network-pre.target
[Install]
RequiredBy=network-pre.target
```

如果系统使用 `ip6tables.service`，那么在 `/etc/systemd/system/ip6tables.service.d` 目录中做同样的配置。更多细节参考 [systemd bug report \(https://bugs.freedesktop.org/show_bug.cgi?id=79600\)](https://bugs.freedesktop.org/show_bug.cgi?id=79600)，[systemd release announcement \(https://lists.freedesktop.org/archives/systemd-devel/2014-June/019925.html\)](https://lists.freedesktop.org/archives/systemd-devel/2014-June/019925.html) 和 [FS#33478 \(https://bugs.archlinux.org/task/33478\)](https://bugs.archlinux.org/task/33478)

通过命令行添加规则，配置文件不会自动改变，所以必须手动保存：

```
# iptables-save > /etc/iptables/iptables.rules
```

修改配置文件后，需要重新加载服务：

```
# systemctl reload iptables
```

或者通过 `iptables` 直接加载：

```
# iptables-restore < /etc/iptables/iptables.rules
```

指南

- [Simple stateful firewall](#)
- [Router](#)

日志

LOG 目标可以用来记录匹配某个规则的数据包。和 ACCEPT 或 DROP 规则不同，进入 LOG 目标之后数据包会继续沿着链向下走。所以要记录所有丢弃的数据包，只需要在 DROP 规则前加上相应的 LOG 规则。但是这样会比较复杂，影响效率，所以应该创建一个 logdrop 链。

创建 logdrop 链：

```
# iptables -N logdrop
```

定义规则：

```
# iptables -A logdrop -m limit --limit 5/m --limit-burst 10 -j LOG
# iptables -A logdrop -j DROP
```

[下文](#)会给出 `limit` 和 `limit-burst` 选项的解释。

现在任何时候想要丢弃数据包并且记录该事件，只要跳转到 `logdrop` 链，例如：

```
# iptables -A INPUT -m conntrack --ctstate INVALID -j logdrop
```

限制日志级别

上述 `logdrop` 链使用限制 (`limit`) 模式来防止 *iptables* 日志来过大或者造成不必要的硬盘读写。没有限制的话，一个试图链接的错误配置服务、或者一个攻击者，都会使 `iptables` 日志写满整个硬盘（或者至少是 `/var` 分区）。

限制模式使用 `-m limit`，可以使用 `--limit` 来设置平均速率或者使用 `--limit-burst` 来设置起始触发速率。在上述 `logdrop` 例子中：

```
iptables -A logdrop -m limit --limit 5/m --limit-burst 10 -j LOG
```

appends a rule which will log all packets that pass through it. The first 10 consecutive packets will be logged, and from then on only 5 packets per minute will be logged. The "limit burst" count is reset every time the "limit rate" is not broken, i.e. logging activity returns to normal automatically.

添加一条记录所有通过其的数据包的规则。开始的连续10个数据包将会被记录，之后每分钟只会记录5个数据包。The "limit burst" count is reset every time the "limit rate" is not broken，例如，日志记录活动自动恢复到正常。

查看记录的数据包

记录的数据包作为内核信息，可以在 [systemd journal](#)^{[[断开的链接](#): 无效的部分]} 看到。

使用以下命令查看所有最近一次启动后所记录的数据包：

```
# journalctl -k | grep "IN=.*OUT=.*" | less
```

使用 syslog-ng

使用 Arch 默认的 [syslog-ng](#) 可以控制 iptables 日志的输出文件：

```
filter f_everything { level(debug..emerg) and not facility(auth, authpriv); };
```

修改为

```
filter f_everything { level(debug..emerg) and not facility(auth, authpriv) and not filter(f_iptables); };
```

iptables 的日志就不会输出到 `/var/log/everything.log` 。

iptables 也可以不输出到 `/var/log/iptables.log`，只需设置 `syslog-ng.conf` 中的 `d_iptables` 为需要的日志文件。

```
destination d_iptables { file("/var/log/iptables.log"); };
```

使用 ulogd

ulogd (<http://www.netfilter.org/projects/ulogd/index.html>) 是专门用于 netfilter 的日志工具，可以代替默认的 LOG 目标。软件包 [ulogd](https://archlinux.org/packages/?name=ulogd) (<https://archlinux.org/packages/?name=ulogd>) 位于 [community] 源。

参见

[Wikipedia:iptables](#)

- [Simple stateful firewall](#)
- [官方网站](http://www.netfilter.org/projects/iptables/index.html) (<http://www.netfilter.org/projects/iptables/index.html>)
- [Iptables 教程 1.2.2](http://www.frozentux.net/iptables-tutorial/iptables-tutorial.html) (<http://www.frozentux.net/iptables-tutorial/iptables-tutorial.html>)

Retrieved from "[https://wiki.archlinux.org/index.php/Iptables_\(简体中文\)&oldid=629211](https://wiki.archlinux.org/index.php/Iptables_(简体中文)&oldid=629211)"

Content is available under [GNU Free Documentation License 1.3 or later](#) unless otherwise noted.